

PDF 5: PERSISTENCIA DE DATOS Y ALMACENAMIENTO LOCAL

Propósito del avance

Hasta el **PDF 4**, los estudiantes han construido una aplicación que puede recibir información, validarla, procesarla y modificar su estado.

Sin embargo, existe un problema:

¿Qué sucede con la información cuando el usuario cierra la aplicación?

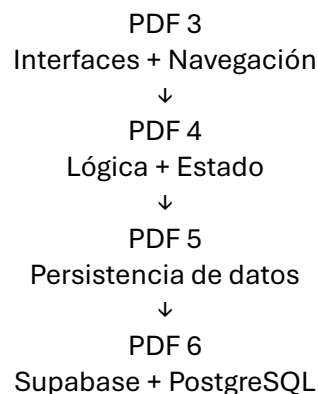
Por ejemplo, si el usuario:

- registra una preferencia,
- agrega un producto al carrito,
- guarda una configuración,
- marca un elemento como favorito,
- registra temporalmente una reserva.

Al cerrar la aplicación, esa información podría perderse.

En este avance deberán comenzar a resolver ese problema mediante la “*persistencia de datos*”.

La evolución será:



1. RETOMAR EL PROYECTO DEL PDF 4

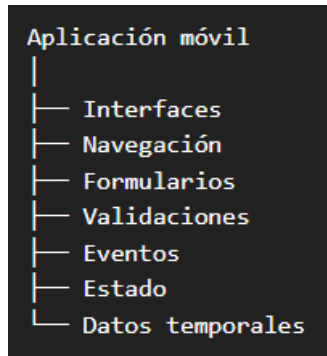
Cada equipo deberá continuar trabajando sobre el mismo proyecto. No deberán crear una aplicación nueva.

Antes de comenzar deberán comprobar:

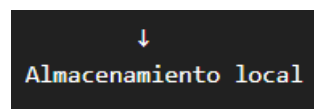
- Las pantallas funcionan.
- La navegación funciona.
- Los formularios funcionan.
- Las validaciones funcionan.
- La funcionalidad principal funciona.

- El repositorio GitHub está actualizado.

La aplicación debería encontrarse aproximadamente en este estado:



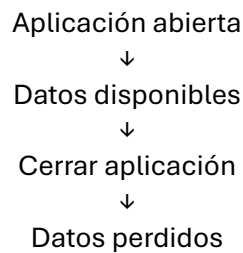
Ahora se incorporará:



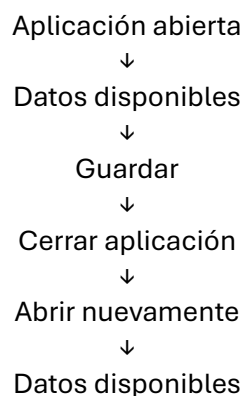
2. COMPRENDER EL CONCEPTO DE PERSISTENCIA

Antes de programar, los estudiantes deberán comprender la diferencia entre:

Datos temporales



Datos persistentes



El objetivo es que comprendan que persistir información significa almacenarla de manera que pueda recuperarse posteriormente.

3. IDENTIFICAR QUÉ INFORMACIÓN NECESITA PERSISTENCIA

Cada equipo deberá revisar su aplicación y determinar qué datos tendría sentido almacenar.

Por ejemplo:

Aplicación de reservas

- Usuario
- Reserva
- Fecha
- Hora
- Cancha seleccionada

Aplicación de farmacia

- Productos favoritos
- Carrito
- Cantidad
- Preferencias

Aplicación de pedidos

- Productos seleccionados
- Carrito
- Dirección
- Preferencias

Aplicación de eventos

- Eventos favoritos
- Eventos seleccionados
- Preferencias del usuario

4. CLASIFICAR LOS DATOS

Cada equipo deberá clasificar la información que maneja su aplicación.

Tipo de información	Ejemplo
Temporal	Resultado de una búsqueda
Persistente local	Preferencias del usuario
Datos de aplicación	Configuración
Datos que posteriormente irán al servidor	Usuarios, reservas, productos

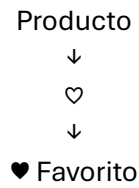
Esto es importante porque no toda la información debe almacenarse de la misma manera.

5. SELECCIONAR UNA FUNCIONALIDAD PARA PERSISTIR

Para este avance cada equipo deberá seleccionar al menos una funcionalidad real de su proyecto que necesite almacenamiento.

Algunas posibilidades:

Opción A - Favoritos



Cerrar aplicación:

Aplicación cerrada

Volver a abrir:

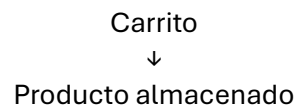


Opción B - Carrito

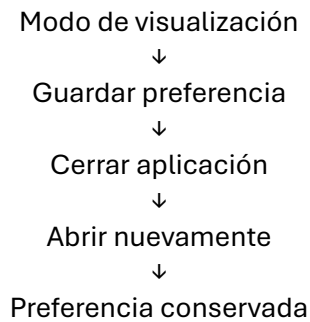


Cerrar aplicación.

Volver a abrir:



Opción C - Preferencias



Opción D - Datos básicos del usuario

Por ejemplo:

Nombre: Juan

Correo: juan@gmail.com

Después de cerrar y abrir:

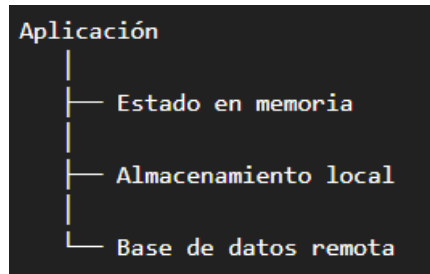
Bienvenido nuevamente, Juan

6. INTRODUCIR EL ALMACENAMIENTO LOCAL

En este avance se trabajará inicialmente con almacenamiento local.

El equipo de desarrollo deberá comprender que existen diferentes mecanismos de almacenamiento.

De manera conceptual:



En este PDF se trabajará principalmente con el segundo, la base de datos remota se abordará posteriormente con *Supabase* y *PostgreSQL*.

7. TRABAJAR CON `shared_preferences`

Para datos sencillos se utilizará:

`shared_preferences`

Este mecanismo permitirá almacenar información simple como:

- Texto.
- Números.
- Booleanos.
- Listas de cadenas.

Por ejemplo:

Nombre → "Juan"

o:

sesionIniciada → true

8. AGREGAR LA DEPENDENCIA

Los estudiantes deberán incorporar la dependencia correspondiente al proyecto.

En *pubspec.yaml* deberán agregar:

```
</> YAML

dependencies:
  shared_preferences: ^2.5.3
```

La versión concreta puede variar según la versión de Flutter disponible en el momento del desarrollo. Lo importante es comprender cómo incorporar y utilizar una dependencia externa.

Después deberán actualizar las dependencias del proyecto.

```
</> Bash  
flutter pub get
```

9. CREAR UN SERVICIO DE ALMACENAMIENTO

Para evitar colocar toda la lógica directamente dentro de las pantallas, deberán comenzar a organizar el código.

Por ejemplo:

```
lib/  
├── screens/  
├── widgets/  
├── models/  
├── services/  
│   └── storage_service.dart  
└── main.dart
```

El archivo:

```
storage_service.dart
```

será responsable de manejar las operaciones de almacenamiento local.

Esto representa un primer acercamiento a una organización más profesional del proyecto.

10. GUARDAR INFORMACIÓN

Los estudiantes deberán implementar una operación que permita guardar información.

Por ejemplo:

```
Usuario introduce nombre  
    ↓  
Presiona GUARDAR  
    ↓  
Aplicación almacena nombre
```

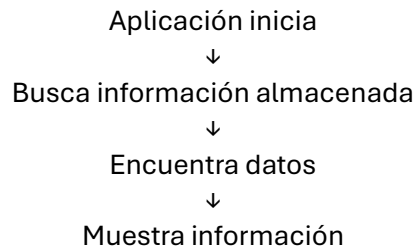
Ejemplo conceptual:

Nombre → Juan

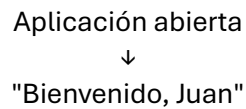
se almacena localmente.

11. RECUPERAR INFORMACIÓN

Después deberán implementar el proceso contrario:



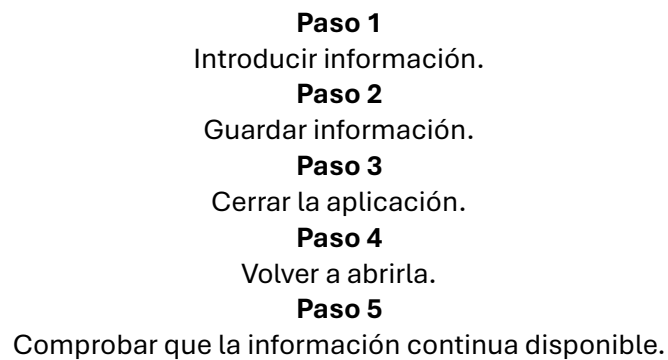
Por ejemplo:



12. COMPROBAR QUE LOS DATOS PERSISTEN

Esta será una de las pruebas más importantes del PDF 5.

Los desarrolladores deberán:



La demostración deberá evidenciar:



13. TRABAJAR CON DATOS BOOLEANOS

También deberán implementar al menos un dato booleano.

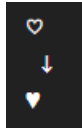
Por ejemplo:

```
sesionIniciada = true
```

o:

```
favorito = true
```

Esto permitirá desarrollar una funcionalidad como:



y mantener ese estado después de cerrar la aplicación.

14. IMPLEMENTAR UNA FUNCIONALIDAD REAL DEL PROYECTO

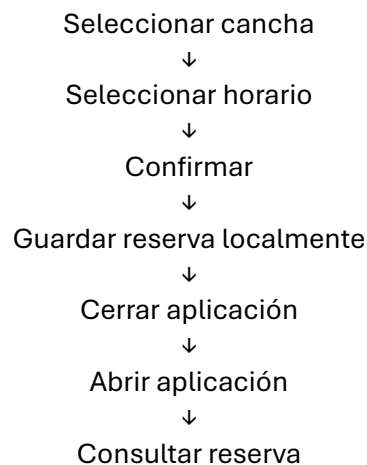
Aquí está la parte más importante del PDF 5.

No se trata simplemente de hacer un ejemplo aislado de `shared_preferences`.

La persistencia deberá incorporarse a la aplicación que los estudiantes están construyendo.

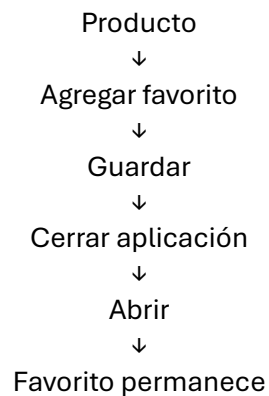
Por ejemplo:

Reserva



O:

Favoritos



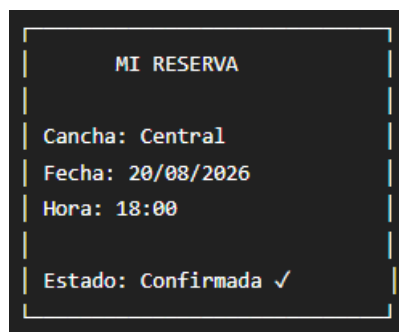
15. CREAR UNA PANTALLA DE INFORMACIÓN PERSISTENTE

Cada equipo deberá mostrar al usuario la información que ha sido almacenada.

Por ejemplo:



O:



16. IMPLEMENTAR ELIMINACIÓN DE DATOS

Además de guardar y recuperar información, deberán implementar al menos una operación de eliminación.

La lógica será:

Guardar
↓
Consultar
↓
Modificar
↓
Eliminar

Por ejemplo:

♥ Favorito
↓
Eliminar de favoritos
↓
♡

Esto introduce una idea fundamental que posteriormente será importante cuando trabajen con bases de datos.

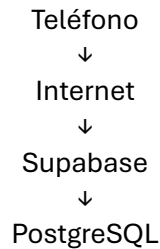
17. DIFERENCIAR ALMACENAMIENTO LOCAL Y BASE DE DATOS

Los estudiantes deberán comprender una diferencia fundamental.

Almacenamiento local



Base de datos remota



En el PDF 5 trabajaremos principalmente con:

Teléfono → almacenamiento local

En el PDF 6 se avanzará hacia:

Aplicación → Supabase → PostgreSQL

18. ANALIZAR LAS LIMITACIONES DEL ALMACENAMIENTO LOCAL

Cada equipo deberá identificar qué información no sería conveniente manejar únicamente de forma local.

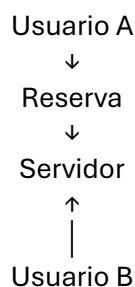
Por ejemplo:

Una aplicación de reservas podría almacenar localmente:

- Preferencias.
- Configuración.
- Información temporal.

Pero una reserva que debe ser conocida por todos los usuarios necesitaría posteriormente un servidor.

Conceptualmente:



Esto permitirá introducir la necesidad de un backend.

19. REALIZAR UNA PRUEBA DE PERSISTENCIA

Cada equipo deberá documentar una prueba real.

Ejemplo

Dato almacenado:

Nombre del usuario: Juan

Proceso:

1. Introducir Juan
2. Guardar
3. Cerrar aplicación
4. Abrir aplicación
5. Recuperar Juan

Resultado esperado:

La aplicación muestra nuevamente el nombre "Juan".

20. REALIZAR PRUEBAS DE ERROR

Deberán comprobar situaciones como:

No existe información: No hay datos almacenados.

Información eliminada: No existen favoritos.

Primera ejecución

Bienvenido.

No se encontraron datos previos.

La aplicación deberá manejar estos casos sin producir errores.

21. ACTUALIZAR LA ESTRUCTURA DEL PROYECTO

La estructura podría quedar:

```
lib/  
├── screens/  
│   ├── splash_screen.dart  
│   ├── login_screen.dart  
│   ├── register_screen.dart  
│   ├── home_screen.dart  
│   └── principal_screen.dart  
├── widgets/  
├── models/  
├── services/  
│   └── storage_service.dart  
└── main.dart
```

Esto representa una evolución respecto al PDF 2 y PDF 3.

22. ACTUALIZAR GITHUB

Una vez implementada la persistencia deberán registrar los cambios.

```
git add .
```

Después:

```
</> Bash  
git commit -m "Implementación de persistencia local"
```

Finalmente:

```
</> Bash  
git push
```

El historial del proyecto debería comenzar a mostrar claramente su evolución:

```
Commit 1  
Inicialización  
  
↓  
  
Commit 2  
Interfaces y navegación  
  
↓  
  
Commit 3  
Lógica y validaciones  
  
↓  
  
Commit 4  
Persistencia local
```

23. ACTUALIZAR EL README

Deberán agregar una sección:

Persistencia de datos

Y explicar:

- Qué información se almacena.
- Por qué se almacena.
- Qué tecnología se utilizó.
- Qué ocurre cuando se cierra la aplicación.
- Qué información puede eliminarse.

Por ejemplo:

La aplicación utiliza almacenamiento local para conservar las preferencias del usuario. La información puede recuperarse al iniciar nuevamente la aplicación y eliminarse cuando el usuario lo solicite.

24. DOCUMENTAR LAS DIFICULTADES

Cada equipo deberá registrar las dificultades encontradas.

Por ejemplo:

Dificultad

La información se guardaba correctamente, pero no se mostraba al volver a abrir la pantalla.

Solución

Se implementó el proceso de lectura de los datos almacenados durante la inicialización de la pantalla.

Otra:

Dificultad

Los datos desaparecían después de cerrar la aplicación.

Solución

Se reemplazó el almacenamiento únicamente en memoria por almacenamiento persistente local.

25. EVIDENCIAS DEL AVANCE

Deberán recopilar evidencias de:

1. Dependencia agregada.
2. pubspec.yaml.
3. Servicio de almacenamiento.
4. Datos guardados.
5. Datos recuperados.
6. Información mostrada nuevamente al iniciar la aplicación.
7. Eliminación de información.
8. Prueba de persistencia después de cerrar y abrir la aplicación.
9. Funcionalidad real del proyecto utilizando almacenamiento local.
10. Repositorio GitHub actualizado.